Anomaly Detection with MLflow

Experiment Report

Capstone Project - Machine Learning Experimentation Workflow

Author: [Your Name]

Date: June 2026

Dataset: Local binary image dataset (data/dataset/no, data/dataset/yes)

MLflow Experiment: anomaly-detection

Tracking URI: http://127.0.0.1:5000

1. Executive Summary

This report documents a complete anomaly detection workflow built for the MLflow capstone assignment. Four model configurations were trained on a local binary image dataset, compared using accuracy, recall (both classes), and macro F1 score, and tracked in a single MLflow experiment. The best model Random Forest was registered, promoted through the model registry (@challenger @champion), and validated with production inference on the held-out test set.

Key finding: Random Forest achieved the highest anomaly recall (74.2%) without SMOTE. Applying SMOTE to XGBoost actually reduced performance on this small, high-dimensional dataset.

2. Introduction

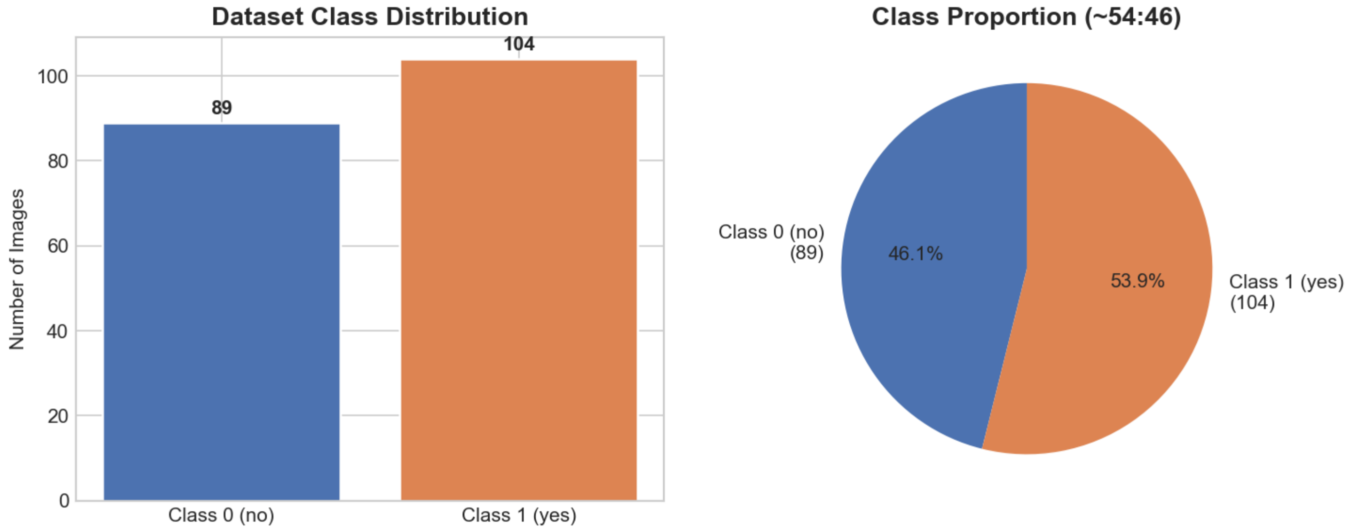
Detecting anomalies in operational data is a common real-world ML challenge. Class imbalance makes evaluation difficult accuracy alone can be misleading when one class dominates. This project implements the required experimentation pipeline: stratified splitting, SMOTE on training data only, four comparable models, MLflow tracking, and model registry workflow.

Dataset: 193 images in two folders no/ (class 0, normal) and yes/ (class 1, anomaly). Images were resized to 32-32 grayscale and flattened into 1,024 numerical features for Logistic Regression, Random Forest, and XGBoost.

3. Dataset and Preprocessing

3.1 Class Distribution

Total: 193 images | Class 0 (no): 89 | Class 1 (yes): 104 | Ratio: ~54:46. The dataset is mildly imbalanced. SMOTE was applied on the training set as required.

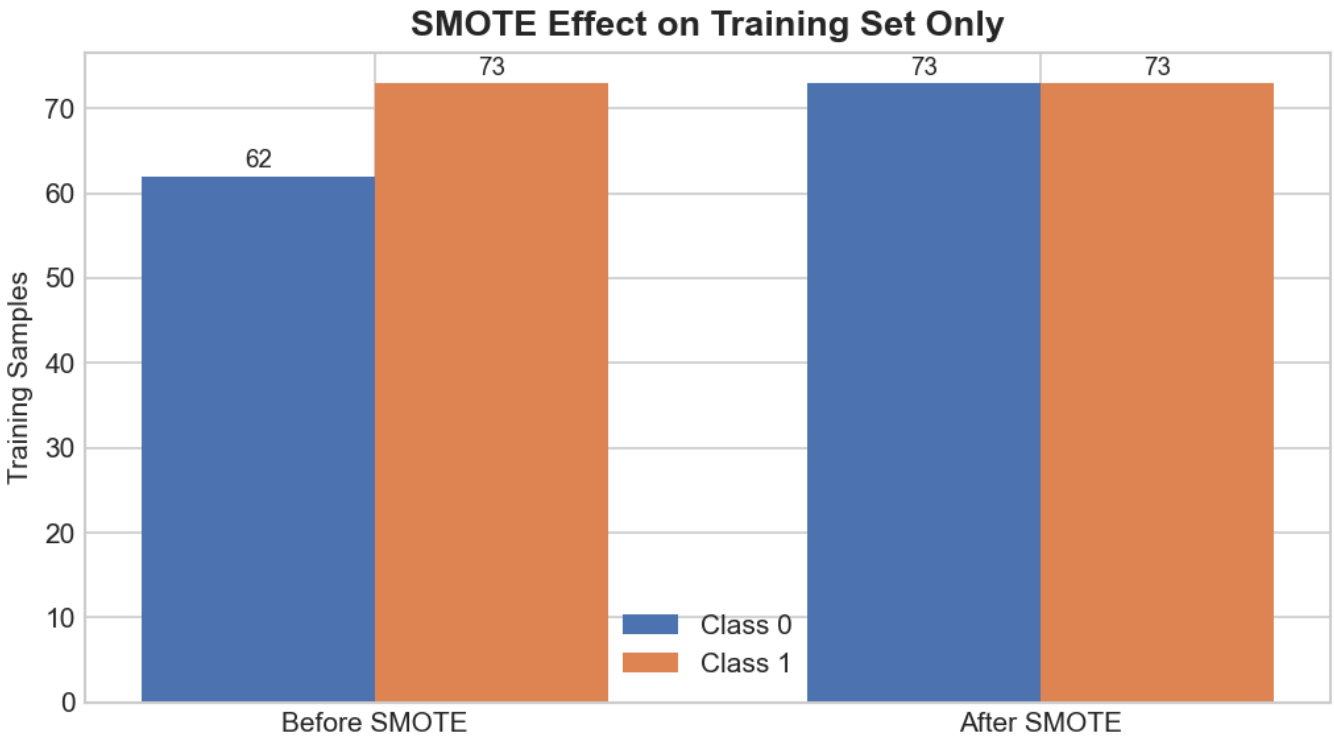


3.2 Train-Test Split (70/30, Stratified)

- Training set: 135 samples
- Test set: 58 samples untouched until final evaluation

3.3 SMOTE on Training Set Only

Before SMOTE: Class 0 = 62, Class 1 = 73. After SMOTE: both classes = 73 (146 total training samples).



4. Experiment Results

4.1 Logistic Regression (Baseline)

Parameters: C=1, solver=liblinear, used_smote=False

Accuracy: 0.4483 | Recall C0: 0.3333 | Recall C1: 0.5484 | F1 Macro: 0.4376
Weakest performer. Linear model struggled with 1,024 pixel features on limited data.

4.2 Random Forest

Parameters: n_estimators=30, max_depth=3, used_smote=False

Accuracy: 0.6207 | Recall C0: 0.4815 | Recall C1: 0.7419 | F1 Macro: 0.6091
Best overall model. Highest recall on anomaly class critical for detection tasks.

4.3 XGBoost (Original Data)

Parameters: eval_metric=logloss, used_smote=False

Accuracy: 0.5690 | Recall C0: 0.5185 | Recall C1: 0.6129 | F1 Macro: 0.5657
Second best. Better than Logistic Regression but below Random Forest.

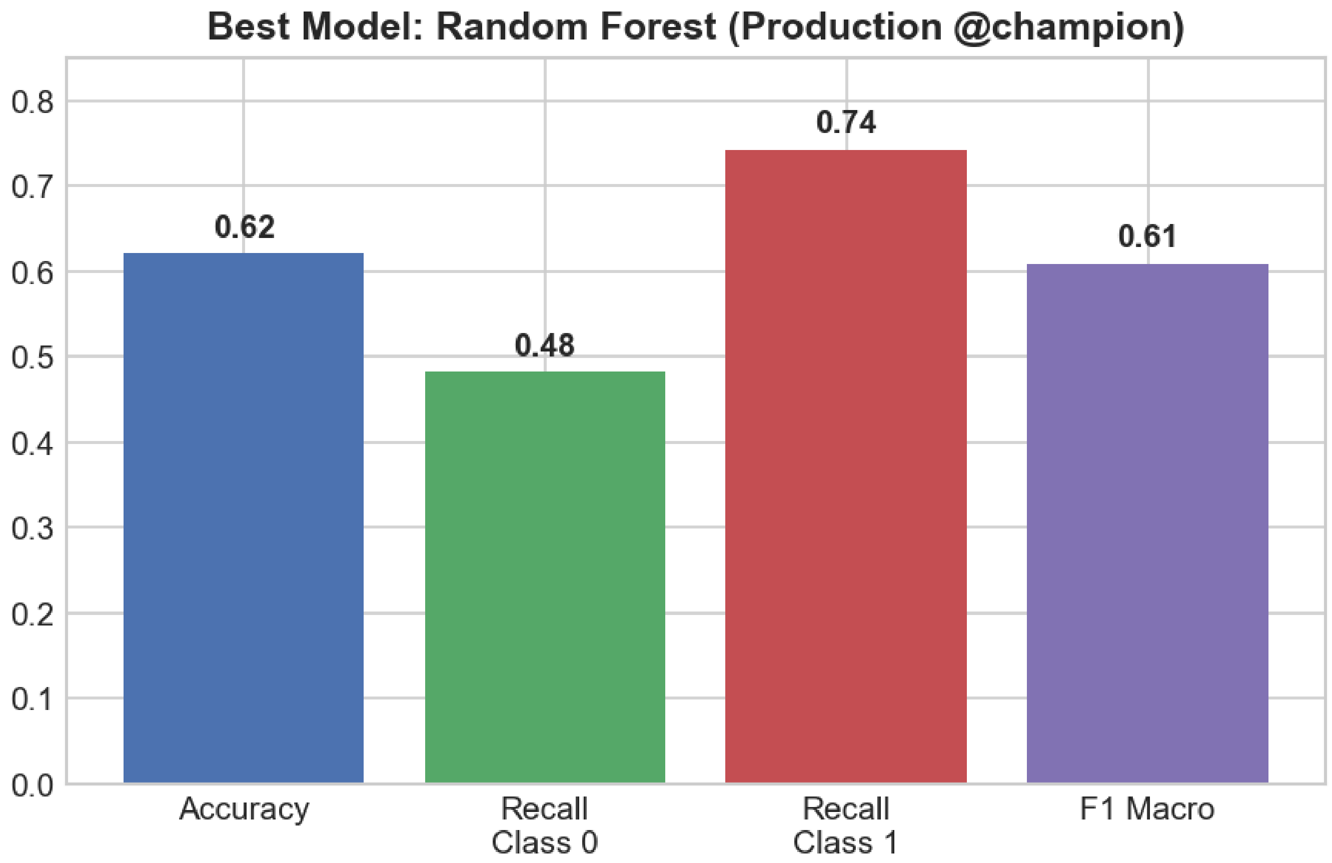
4.4 XGBoost with SMOTE

Parameters: eval_metric=logloss, used_smote=True

Accuracy: 0.5000 | Recall C0: 0.4815 | Recall C1: 0.5161 | F1 Macro: 0.4987
SMOTE hurt performance. Oversampling added noise on this small high-dimensional set.

5. Metric Comparison

Model	Accuracy	Recall C0	Recall C1	F1 Macro	SMOTE
Random Forest	0.6207	0.4815	0.7419	0.6091	No
XGBClassifier	0.5690	0.5185	0.6129	0.5657	No
Logistic Regression	0.4483	0.3333	0.5484	0.4376	No
XGBoost + SMOTE	0.5000	0.4815	0.5161	0.4987	Yes



6. Observations on Class Imbalance Handling

- Dataset imbalance is mild (~54:46), so sample size and feature dimension mattered more than extreme skew.
- SMOTE balanced the training set (6273 for class 0) but did not improve XGBoost test performance.
- Random Forest without SMOTE achieved the best anomaly recall tree ensembles handled pixel patterns better.
- Selection criterion: recall_class_1 first, then f1_score_macro appropriate when missing anomalies is costly.
- For production image tasks, a CNN would likely outperform raw pixels; this workflow meets capstone MLflow requirements.

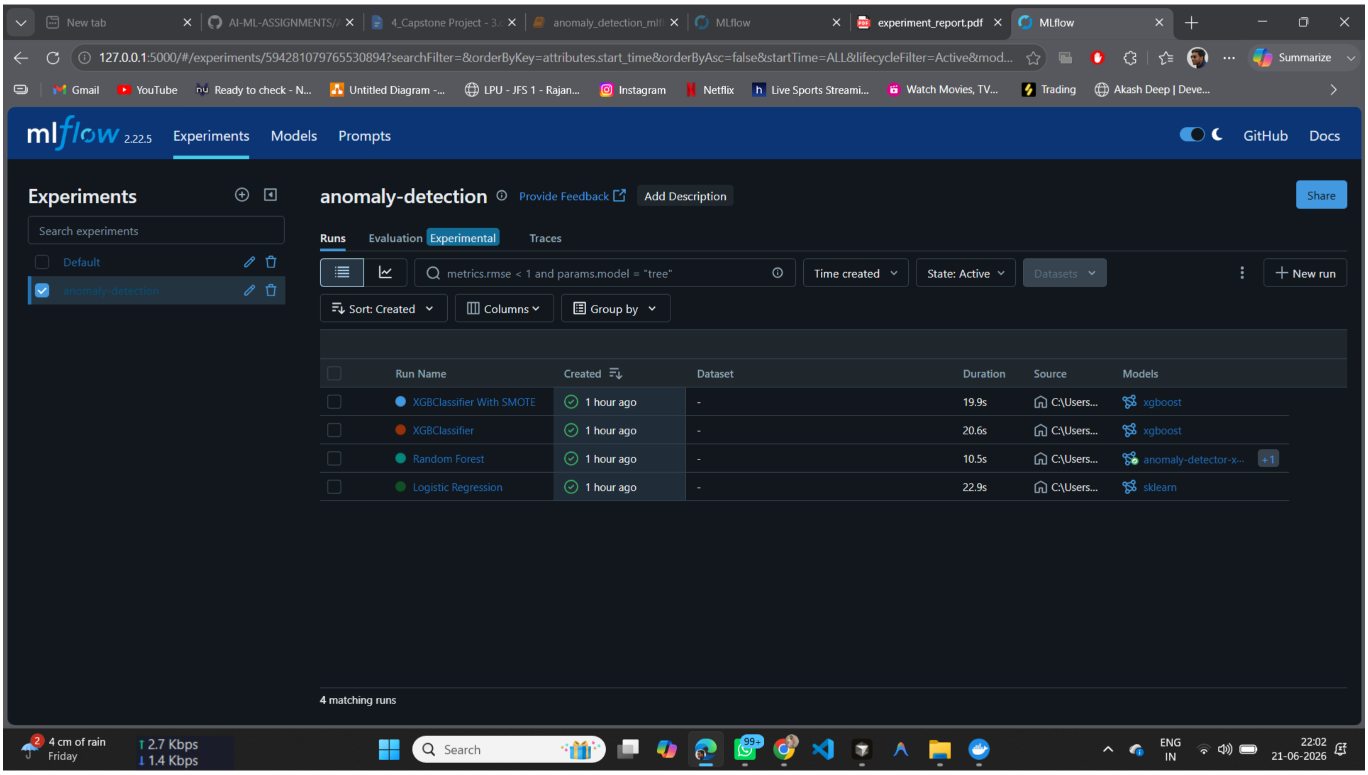
7. MLflow Tracking and Model Registry

All runs logged to experiment 'anomaly-detection' on http://127.0.0.1:5000. Each run recorded hyperparameters, four metrics, and model artifacts.

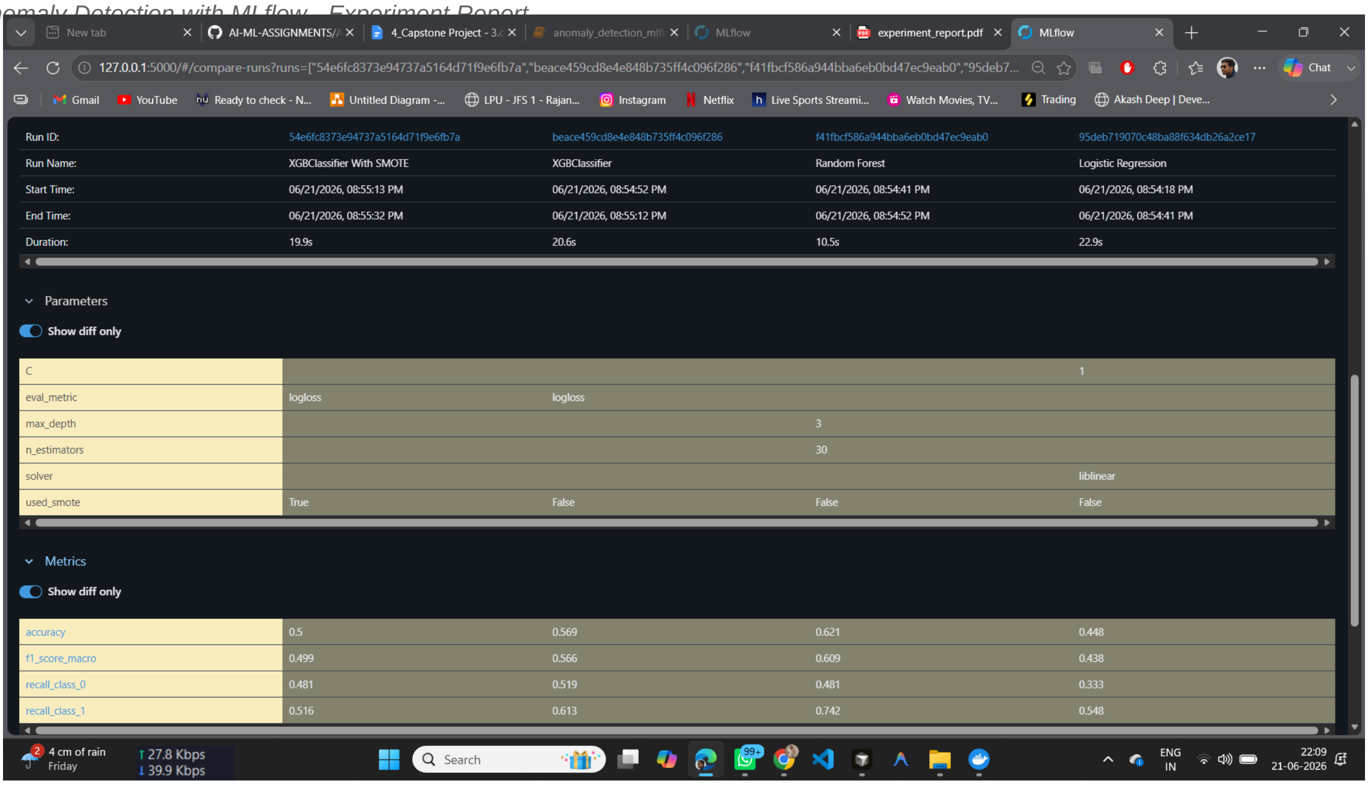
Registry Workflow

- Registered best model as anomaly-detector-xgb-smote v1
- Assigned @challenger alias
- Copied to anomaly-detection-prod v1
- Assigned @champion alias
- Loaded models:/anomaly-detection-prod@champion test metrics matched Random Forest

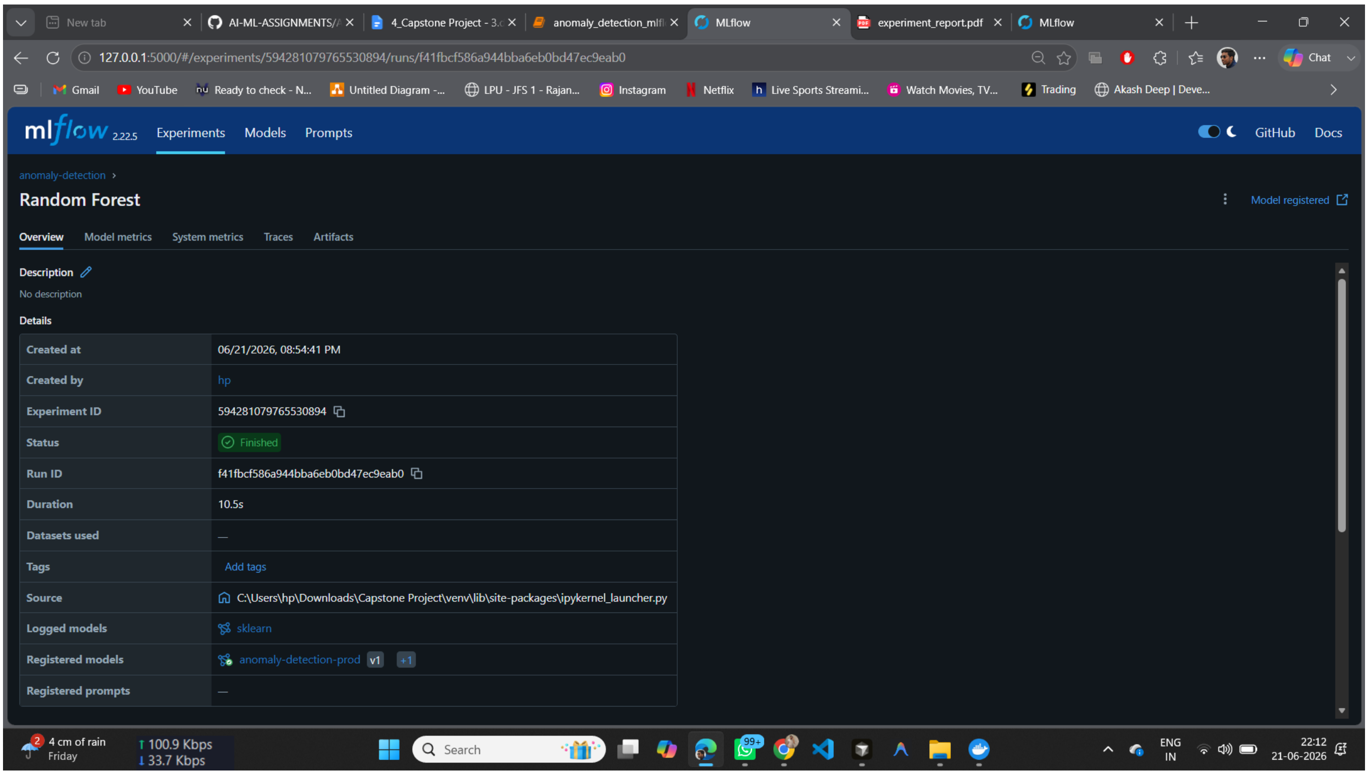
7.1 Experiments List View



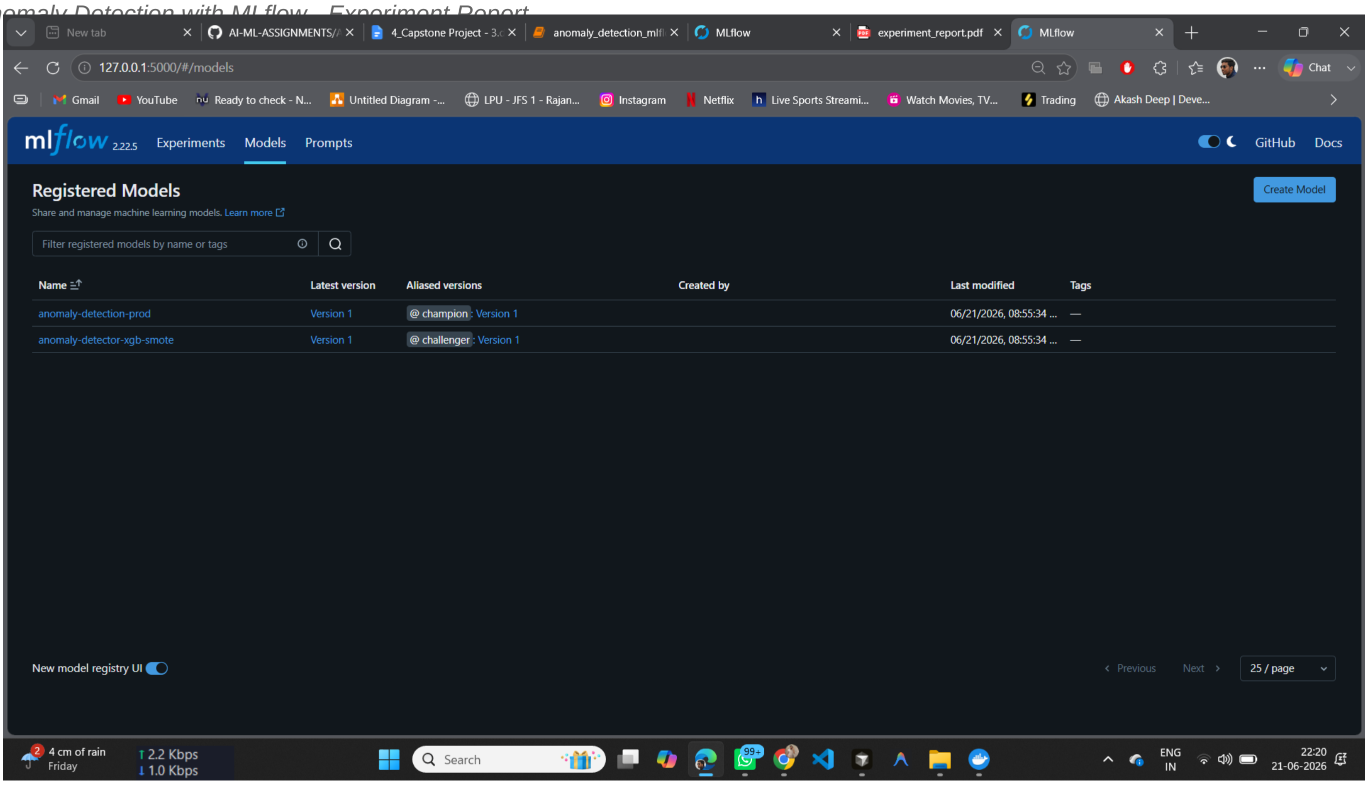
7.2 Runs Comparison View



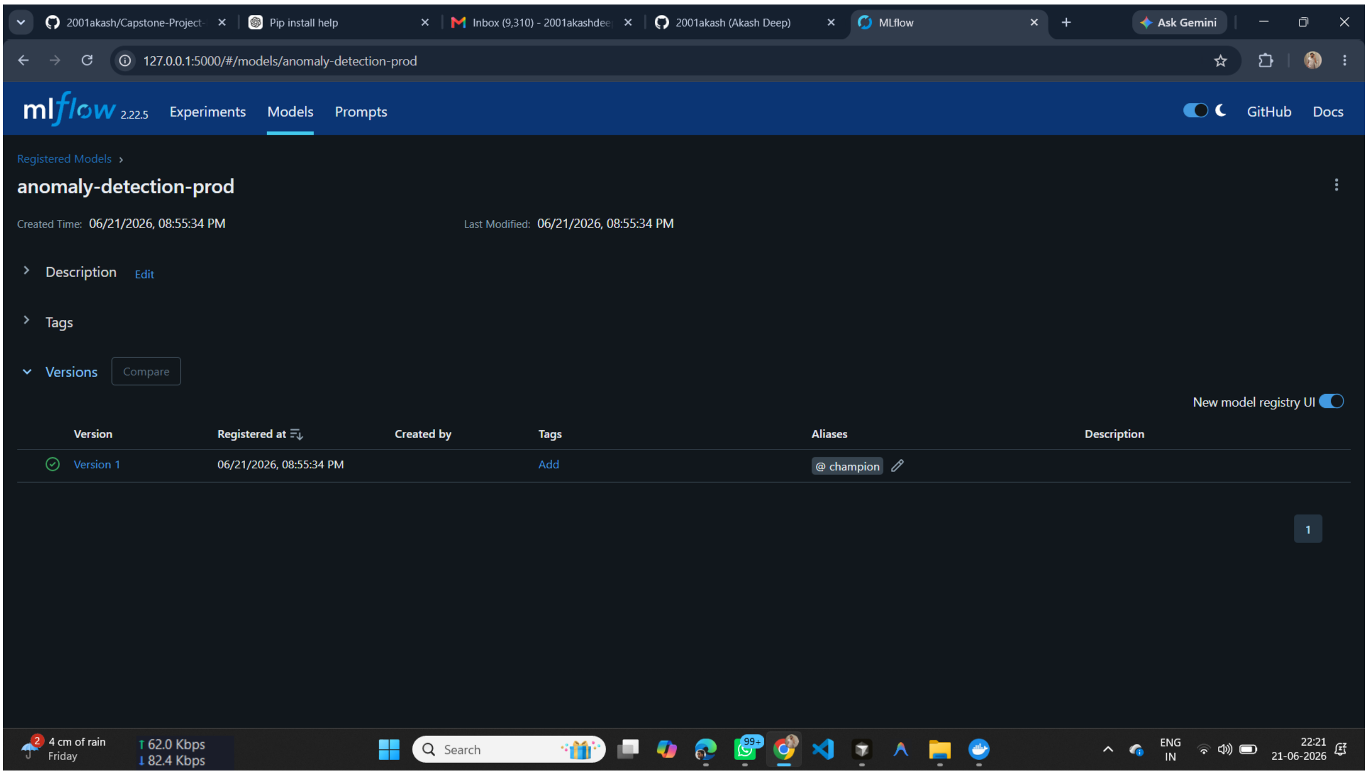
7.3 Best Run Detail Random Forest



7.4 Model Registry @challenger



7.5 Production Model @champion



8. Production Model Justification

Production model: Random Forest deployed as anomaly-detection-prod @champion.

Justification: Random Forest ranked first on both primary selection metrics recall on class 1 (0.7419) and macro F1 (0.6091). It correctly identifies ~74% of anomalies on the test set while maintaining the highest overall balance across classes. Production inference confirmed identical metrics, validating the end-to-end MLflow registry pipeline.

9. Conclusion

Four experiments were successfully tracked, compared, and registered in MLflow. Random Forest was selected for production. SMOTE was valuable to document but not beneficial for XGBoost on this dataset. The workflow demonstrates reproducible experimentation, metric-driven model selection, and production-ready model deployment through MLflow model registry.

10. References

- Dataset: Local repository data/dataset/no/, data/dataset/yes/
- MLflow: https://mlflow.org/docs/latest/
- imbalanced-learn SMOTE: https://imbalanced-learn.org/